

Unit Testing Method Naming Conventions

- **[MethodName]_[Scenario]_[ExpectedResult]**: This is a highly descriptive and common pattern.

Example: `divide_byZero_throwsException()`

- **[UnitOfWork]_[StateUnderTest]_[ExpectedBehavior]**: Similar to the above, popular in enterprise testing.

Example: `isAdult_ageLessThan18_returnsFalse()`

- **BDD Style (Given_When_Then)**: Aligned with Behavior-Driven Development (BDD), this pattern explicitly outlines the preconditions, the action, and the outcome.

Example: `Given_UsersIsAuthenticated_When_InvalidAccountNumberIsUsedToWithdrawMoney_Then_TransactionWillFail()`

- **should[ExpectedBehavior]When[StateUnderTest]**: Uses a "should" prefix to emphasize the expected behavior.

Example: `shouldReturnTrueWhenNumberIsEven()`

Behavior-Driven Development (BDD)

In Behavior-Driven Development (BDD), the **Given-When-Then** (GWT) pattern is a structured format for writing acceptance criteria and scenarios in plain language, facilitating clear communication among technical and non-technical team members. It breaks down system behavior into a simple, logical flow of **context**, **action**, and **expected outcome**.

The GWT format is often used with the Gherkin domain-specific language, which employs keywords like **Given**, **When**, **Then**, **And**, and **But** to structure scenarios that can be automated using tools like Cucumber or SpecFlow.

The Components of Given-When-Then

Each part of the structure serves a specific purpose:

- **Given**: This section describes the **initial context or preconditions** of the system before the action occurs. It sets the scene and puts the system into a known, consistent state. Multiple preconditions can be specified using the **And** keyword.

- *Example:* Given the user is on the login page
- **When:** This section describes the **specific action or event** that triggers the behavior being described. It is the interaction with the system (e.g., a user clicking a button or a system event occurring) and should generally focus on a single behavior per scenario.
- *Example:* When they enter valid credentials and click "Log In"
- **Then:** This section describes the **expected outcome or result** of the action taken in the When clause. It defines the post-conditions or observable changes that validate the behavior is working as intended.
- *Example:* Then they should be successfully redirected to their account dashboard

A Full Scenario Example

Here is a complete example of a BDD scenario using the GWT format:

gherkin

Scenario: Successful account login

Given the user is on the login page

When the user enters their valid email and password

And clicks the "Sign In" button

Then the user should be logged into their account

And the user's dashboard should be displayed

Eg:

Feature: User trades stocks

Scenario: User requests a sell before close of trading

Given I have 100 shares of MSFT stock

And I have 150 shares of APPL stock

And the time is before close of trading

When I ask to sell 20 shares of MSFT stock

Then I should have 80 shares of MSFT stock

And I should have 150 shares of APPL stock

And a sell order for 20 shares of MSFT stock should have been executed

Arrange-Act-Assert (AAA) pattern

The Arrange-Act-Assert (AAA) pattern is a standardized, three-step structure for organizing unit tests to improve readability and maintainability. It breaks a test into: **Arrange** (setting up data/dependencies), **Act** (executing the specific method), and **Assert** (verifying the outcome). This approach ensures tests are focused and clean.

Key Components of the AAA Pattern:

- **Arrange:** In this phase, initialize objects, create mock data, set up dependencies, or configure the environment required for the scenario.
- **Act:** This phase invokes the actual function, method, or operation being tested.
- **Assert:** This final phase verifies that the action performed in the "Act" step produced the expected outcome, typically using equality checks or boolean verifications.